parallel bfs explanation

13 May 2026 01:40

HPC Assignment 1 — Parallel BFS using OpenMP

This assignment is:

Design and implement Parallel Breadth First Search (BFS) using OpenMP

using:

- graph traversal
- parallel programming
- OpenMP directives

1. What We Have To Do in This Assignment?

We have to:

1. create a graph
2. traverse graph using BFS
3. apply parallelism using OpenMP
4. process nodes simultaneously

Main Goal

To make BFS:

faster using parallel processing

instead of purely sequential execution.

2. What is BFS?

BFS =

Breadth First Search

Graph traversal algorithm.
It visits:

level by level

Example

Graph:

```
    0
   / \
  1   2
 / \   \
3   4   5
```

BFS Traversal from 0:
0 1 2 3 4 5

Why “Breadth” First?

Because:

- first all neighbors visited
- then next level neighbors

3. What is OpenMP?

OpenMP =

Open Multi-Processing

Used for:

parallel programming in C/C++

Purpose

Allows:

- multiple threads
- simultaneous execution
- multicore CPU utilization

Important OpenMP Directive

#pragma omp parallel for

This converts loop into:

parallel loop

Multiple threads execute iterations simultaneously.

4. Overall Workflow of Program

Input Graph

↓

Choose Starting Node

↓

Visit Current Level Nodes

↓

Parallel Processing of Neighbors

↓

Store Next Level

↓

Repeat Until Finished

5. How Parallel BFS Works Here?

Normally BFS:

- processes one node at a time

Parallel BFS:

- processes multiple nodes in current level simultaneously

Example

Current level:

1 2 3

Thread 1:

process node 1

Thread 2:

process node 2

Thread 3:

process node 3

at same time.

6. Code Explanation Line by Line

Header Files

#include <iostream>
#include <vector>
#include <omp.h>

Purpose

Header	Purpose
iostream	input/output
vector	dynamic arrays
omp.h	OpenMP functions

Namespace

using namespace std;
Avoids writing:
std::
again and again.

Main Function

int main()
Program execution starts here.

Input Number of Vertices

int n;
cout << "Enter number of vertices: ";
cin >> n;
User enters:

total nodes in graph

Create Adjacency Matrix

vector<vector<int>> graph(n, vector<int>(n));
Creates:

2D matrix

What is Adjacency Matrix?

Graph representation.
Example:
0 1 1
1 0 1
1 1 0
Meaning:

- 0 connected to 1 and 2

Input Graph

for (int i = 0; i < n; i++) {

```
    for (int j = 0; j < n; j++) {
        cin >> graph[i][j];
    }
}
```
Reads adjacency matrix.

Starting Vertex

```
int start;
cout << "Enter starting vertex: ";
cin >> start;
```
Starting point for BFS traversal.

Visited Array

```
vector<bool> visited(n, false);
```
Tracks:

which nodes already visited

Initially:
all false

Current and Next Level

```
vector<int> current_level, next_level;
```
Purpose:

Vector	Meaning
current_level	nodes currently processing
next_level	nodes for next BFS level

Mark Start Node Visited

```
visited[start] = true;
current_level.push_back(start);
```
Starting node:

- marked visited
- added to BFS queue

BFS Traversal Output

```
cout << "BFS Traversal: ";
```

Main BFS Loop

```
while (!current_level.empty())
```
Loop continues until:

no nodes left

Clear next_level

```
next_level.clear();
```
Very important.
Removes old level nodes before storing new ones.

Print Current Level Nodes

```
for (int node : current_level) {
   cout << node << " ";
}
```
Displays traversal order.

Parallel Region

#pragma omp parallel for
MOST IMPORTANT LINE.
Converts loop into:

parallel execution

Different threads process different nodes.

Process Each Current Node

for (int i = 0; i < (int)current_level.size(); i++)
Each thread processes:

- one node from current level

Current Node

int u = current_level[i];
Current processing node.

Traverse Neighbors

for (int v = 0; v < n; v++)
Checks all adjacent vertices.

Check Edge Exists

if (graph[u][v] == 1)
Means:

edge exists between u and v

should_add Variable

bool should_add = false;
Used to safely decide:

- whether node should enter next level

Critical Section

#pragma omp critical
MOST IMPORTANT PARALLEL CONCEPT.
Only ONE thread enters critical section at a time.

Why Needed?

Multiple threads may:

- access visited array simultaneously

Without critical:

race condition occurs

Race Condition

Suppose:

- Thread 1 and Thread 2 both check same node

Both may insert duplicate node simultaneously.
Critical prevents this.

Visit Node Safely

```
if (!visited[v]) {
   visited[v] = true;
   should_add = true;
}
```

If node unvisited:

- mark visited
- allow insertion into next level

Add to Next Level

```
next_level.push_back(v);
```

Stores node for future BFS processing.
Also inside critical section for thread safety.

Move to Next Level

```
current_level = next_level;
```

After current level processed:

next level becomes current level

End Program

```
return 0;
```

7. Example Execution

Suppose graph:

```
0 -- 1
|    |
2 -- 3
```

Adjacency matrix:

```
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
```

Start:

```
0
```

Iteration 1

Current level:

```
[0]
```

Visit:

```
1 2
```

Next level:

```
[1,2]
```

Iteration 2

Current level:

[1,2]
Visit:

3

Next level:
```text id="u2'wini"
[3]

# Final BFS
0 1 2 3

# 8. How Parallelism Happens?
Suppose current level:
[1,2]
Thread 1:
process neighbors of 1
Thread 2:
process neighbors of 2
simultaneously.
This improves speed.

# 9. Compilation Command
g++ -fopenmp bfs.cpp -o bfs

# Run
./bfs

# Why -fopenmp Needed?
Enables:
# OpenMP support
Without it:
#pragma omp
ignored.

# 10. Important Viva Questions
## Q1. What is BFS?
Breadth First Search is a graph traversal algorithm that visits nodes level by level.

## Q2. Why queue-like behavior used in BFS?
Because BFS processes nodes in level order.

## Q3. What is OpenMP?
API for parallel programming in C/C++.

## Q4. What does #pragma omp parallel for do?
It parallelizes loop iterations among multiple threads.
```

Q5. What is race condition?
When multiple threads access shared data simultaneously causing incorrect behavior.

Q6. Why critical section used?
To prevent race conditions during shared data updates.

Q7. What is adjacency matrix?
2D matrix representation of graph connections.

Q8. Why visited array used?
To avoid revisiting nodes.

Q9. What is current_level and next_level?
Used to process BFS level-by-level.

Q10. Why clear next_level?
To remove old nodes before storing next BFS level.

11. Time Complexity
Sequential BFS:
O(V+E)
Where:
- (V) = vertices
- (E) = edges

Parallel BFS improves practical execution speed using multiple threads.

Time Complexity of Parallel BFS
For normal sequential BFS:
$O(V + E)$
Where:
- V = number of vertices
- E = number of edges

Parallel BFS Time Complexity
In parallel BFS using:

#pragma omp parallel for
multiple nodes of same level are processed simultaneously.
So theoretical complexity becomes approximately:
$$O\left(\frac{V + E}{P}\right)$$
Where:
- P = number of processors/threads

Meaning
Suppose:

Value	Meaning
$V + E = 1000$	total work
$P = 4$	4 threads

Then each thread handles approximately:

1000 / 4 = 250 operations
So execution becomes faster.

Important Reality

This is:

ideal theoretical complexity

Actual performance depends on:

- synchronization overhead
- critical sections
- thread management
- graph structure